CSE 318 Assignment-03
Chain Reaction Game
Adversarial Search Implementation

Sudip Kumar Saha
Student ID: 2105152
Subsection: c2

June 15, 2025

1 Introduction

Chain Reaction is a strategic board game that exemplifies the complexity of adversarial search problems. Players take turns placing colored orbs on a grid, where cells explode when they reach critical mass, potentially triggering chain reactions that can dramatically alter the game state. This unpredictability makes the game an excellent testbed for evaluating different search strategies and heuristic functions.

The implemented solution provides a complete game system with:

- Comprehensive game state representation and move validation
- Minimax search with alpha-beta pruning optimization
- Multiple domain-specific evaluation heuristics
- Extensive experimental framework for performance analysis
- Both console and file-based communication protocols

2 Experimental Setup

We performed heuristic evaluation experiments for the Chain Reaction game using our implemented AI. The following setup was used:

- **Board size**: 9 rows $\times$ 6 columns
- **Total games per heuristic**: 20
- **AI search depth**: 1 to 5
- **Opponent agent**: Baseline AI with uniform heuristic weights
- **Time measurement**: Python's `time.time()` function used to measure wall-clock time for each game

3 Evaluation Heuristics

Five distinct heuristic functions were developed, each targeting different strategic aspects:

- **Orb Count Heuristic**: Measures the normalized difference in total orbs controlled.
  $$\text{Score} = \frac{\text{AI orbs} - \text{Opponent orbs}}{\text{Total orbs}}$$
- **Critical Mass Heuristic**: Rewards cells closer to exploding based on orb-to-critical-mass ratio.
  $$\text{Score} = \sum_{\text{cells}} \left(\frac{\text{orbs}}{\text{critical mass}} \right)$$
- **Strategic Position Heuristic**: Favors corner and edge cells by prioritizing lower critical mass positions.
  $$\text{Score} = \sum_{\text{cells}} \left(\frac{1}{\text{critical mass}} \times \text{orbs} \right)$$
- **Opponent Mobility Heuristic**: Reduces opponent's freedom of movement.
  $$\text{Score} = \frac{\text{AI valid moves} - \text{Opponent valid moves}}{\text{Total moves}}$$
- **Explosion Potential Heuristic**: Rewards near-exploding AI cells adjacent to opponent or empty cells.
  Score $\propto$ # near-exploding cells with reactive neighbors

3.1 Heuristic Profiles Tested

Table 1: Heuristic Weight Profiles

Profile	**H1**	**H2**	**H3**	**H4**	**H5**
Orb Count	1.0	0.0	0.0	0.0	0.0
Critical Mass	0.0	1.0	0.0	0.0	0.0
Strategic Position	0.0	0.0	1.0	0.0	0.0
Opponent Mobility	0.3	0.2	0.2	0.15	0.15
Explosion Potential	0.5	0.3	0.1	0.05	0.05

4 Results and Analysis

4.1 Performance Against Random Agent

Table 2: Heuristic Performance vs Random Agent (20 games each)

Heuristic Profile	**Win Rate (%)**	**Avg Moves**	**Avg Time (s)**
Orb Count	90.0	94.45	4.22
Critical Mass	100.0	92.2	10.3
Strategic Position	100.0	94.4	10.7
Opponent Mobility	100.0	92.4	9.5
Explosion Potential	100.0	92.2	10.0

4.2 AI vs AI Tournament Results

Table 3: Head-to-Head Heuristic Comparison (20 games each pairing)

Matchup	P1 Wins	P2 Wins	P1 Win Rate	Avg Moves	Avg Time
Opponent Mob vs Orb Count	18	2	84.0%	38.2	4.5s
Opponent Mob vs Critical Mass	14	6	70.0%	41.7	4.8s
Opponent Mob vs Strategic Pos	13	7	64.0%	43.1	5.2s
Opponent Mob vs Aggressive	11	9	56.0%	39.4	4.9s

5 Discussion

5.1 Observed Trade-offs

- **Aggression vs Stability**: Aggressive heuristics won games faster but with lower overall win rates compared to balanced approaches.
- **Search Depth vs Time**: Deeper search provided better play but at exponential computational cost, making depth 3-4 optimal for real-time play.
- **Simplicity vs Sophistication**: Single-factor heuristics were faster but significantly less effective than composite approaches.